A Baseline for Integer Factorization Performance

Maren Peasley
chroisa@gmail.com

ABSTRACT. This paper provides a partial baseline for integer factorization with the CADO-NFS package.
KEYWORDS integer factorization, cado-nfs, rsa, baseline

1 INTRODUCTION

This effort provides a partial baseline for integer factorization with the CADO-NFS v2.3.0 package, using the parameters provided in that package. Complete factorizations for five trials each were performed for composite integers from 30-100 digits. Polynomial selection only was performed for composite integers from 105-190 digits (five trials each). Complete data sets, as well as summaries of the data + charts, are linked below:

Summary Results spreadsheet (XLSX, < 1 MB): https://github.com/Chroisa/A-Baseline-for-Integer-Factorization/raw/refs/heads/master/intfactcharts-en.xlsx

Complete data set (ZIP, ~80 GB): https://github.com/Chroisa/A-Baseline-for-Integer-Factorization/blob/master/README.md

2 BACKGROUND

Integer factorization, apart from mathematics, enjoys a special role in modern security in its use by the Rivest-Shamir-Adleman ("RSA") algorithm. The difficulty of integer factorization provides the security level of RSA. The currently favored algorithm for general-purpose integer factorization is "GNFS" or General Number Field Sieve. For further background, the reader is encouraged to read "A Tale of Two Sieves" (see the References section for more info), by Carl Pomerance.

Two well-known software packages exist that implement GNFS: msieve and CADO-NFS. The author selected CADO-NFS due to its ease in operating in distributed computation environments and familiarity with the software.

GNFS itself is well studied and is split into four operating phases: polynomial selection, lattice sieving, filtering, and linear algebra. Subphases specific to the CADO-NFS software are listed under the major

phases, below. For reporting convenience, quadratic characters and the final square root were included in the "Linear Algebra" phase.

- Polynomial Selection
  - Size optimization
  - Root optimization
  - Generate factor base
  - Generate free relations
- Lattice Sieving
- Filtering
  - Duplicate removal, splitting pass
  - Duplicate removal, removal pass
  - Singleton removal
  - Merging – merge
  - Merging – replay
- Linear Algebra
  - Linear Algebra
  - Quadratic characters
  - Square root

CADO-NFS provides a set of parameters for individual composite integers ranging from 30 digits long to 320 digits long. There are many non-trivial unknowns about wielding GNFS, including:

- Estimated CPU time necessary to factor a non-prime integer of a specific size
- Estimated wall clock time necessary to factor a non-prime integer of a specific size
- Disk space needed to factor a non-prime integer of a specific size
- Memory needed to factor a non-prime integer of a specific size
- Number of compute nodes necessary

This work provides a partial answer to the above questions while also referring to a new, publicly available dataset and summary results tested against the latest version of CADO-NFS available at the time of writing (version 2.3.0).

2.1 TERMINOLOGY

This paper refers to the size of composite integers in terms of the count of characters of its decimal representation. This is done to correspond closely with the CADO-NFS package. A table showing character count of composite integers alongside some example sizes as represented in bits for select values is given below:

Characters	Bits
30	98 – 99
60	197 – 199
90	297 – 299
155	512 – 514
190	629 – 631
310	1027 – 1030

Approximations are made throughout this paper for reference. This information is also reflected in the "Index of Integers" tab of the Summary Results spreadsheet, linked at the top of this paper.

A more precise method is to compare the smallest and largest decimal values as represented in binary:

100..000 (29 zeroes, decimal) = 101000…000 (97 binary digits)
999…999 (30 nines, decimal) = 110010…111 (100 binary digits)

3 INTEGER GENERATION

Several methods were used to generate composite integers of the correct length. "Length" determined here by a simple count of the number of decimal digits. A summary of the process is shown below:

For a composite 30 characters in length (~99 bits),

1) Generate two primes of 15 characters,
2) Multiply these together and represent them in decimal,
3) Verify that the result is 30 characters long.

Prime generation followed several methods:

A) Guess-and-Check
   Check if JavaScript-generated integer sequence of 15 digits was prime.
   This was an expensive use of the primality checker.
B) Find next probable prime lower
   a. Generate an integer sequence of 15 characters
   b. Use the primality checker to generate the next probable prime lower than this
   c. Use the primality checker to verify that this generated value is indeed a prime
   d. Multiply two generated 15-digit primes together and save the result.
C) Find next probable prime higher
   a. Generate an integer sequence of 15 characters
   b. Use the primality checker to generate the next probable prime higher than this
   c. Use the primality checker to verify that this generated value is indeed a prime
   d. Multiply two generated 15-digit primes together and save the result.

Method "A" was used initially until it became a nuisance. Method "B" was used until it generated numbers that converted into N-1 digit composite integers. Method "C" used for the remainder.

Integers can be found in the Summary Results spreadsheet (linked at the top) in a tab entitled "Index of Integers".

Note that two primes of similar size were used as this most closely resembles the security application – that integer factorization is hard. Other forms exist but were not studied in this analysis.

The primality checker used is available online at https://www.alpertron.com.ar/ECM.HTM and performs probabilistic primality testing using Baillie-PSW primality testing.

4 FACTOR PAGES

Parameters for each of the four phases as well as results from five trials (unique composites from unique primes) are listed on one tab in the Summary Results spreadsheet referred to as a "Factor page". Each tab has a section for navigation, polynomial selection, lattice sieving, filtering, and linear algebra, as well as additional selected data.

Examples of additional data included: "MurphyE" score of polynomial selected from the polynomial selection phase, wall clock time overall or for just the polynomial selection, summary of compute nodes used for processing these composite integers.

Each tab has titles such as "c30" (for 30-digit composite integers) or "c320" (for 320-digit composite integers, or ~1028 bits).

Additional care was taken in the formatting and presentation of the data – so additional explanation of the data is likely not necessary.

5 PARAMETER SELECTION

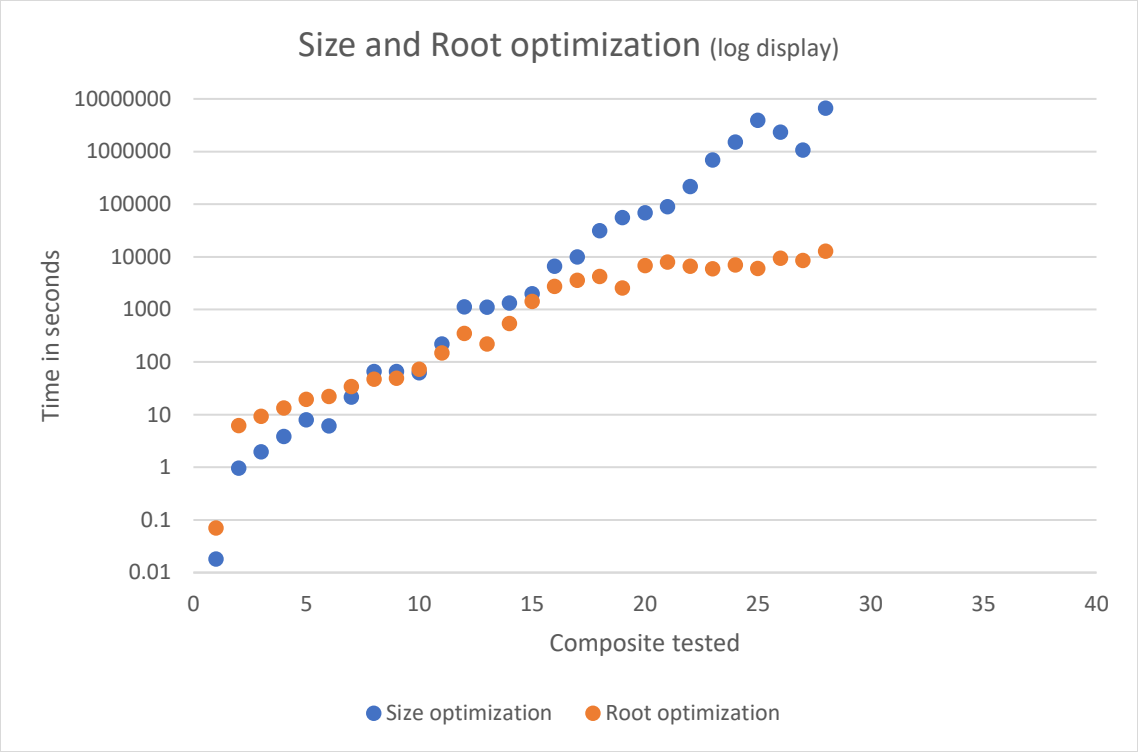
Parameters were left at their defaults from the CADO-NFS v2.3.0 package. For reference, these are given in a summary of the data in the "Factor pages", titled "c30", "c60", "c320", and similarly.

Parameters are given on each "Factor page" as well as summarized in an "Index of Parameters". The "Index of Parameters" also displays some of the parameters logarithmically in bar graphs as a convenience.

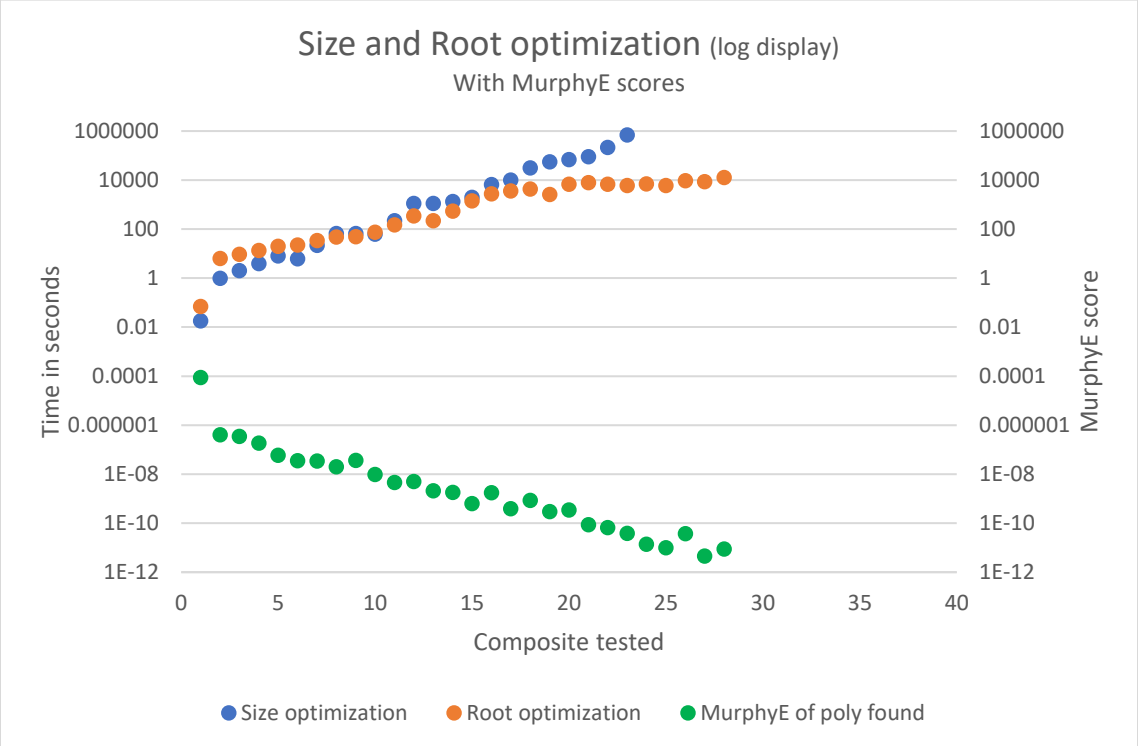
6 RESULTS

Complete factorization was only performed for composites ranging from 30 digits to 110 digits, due to resource constraints. Polynomial selection only (size optimization, root optimization) was performed for composites ranging from 115 digits (~381 bits) to 190 digits (~630 bits).

Size optimization and root optimization both appear to grow exponentially, but at different rates:

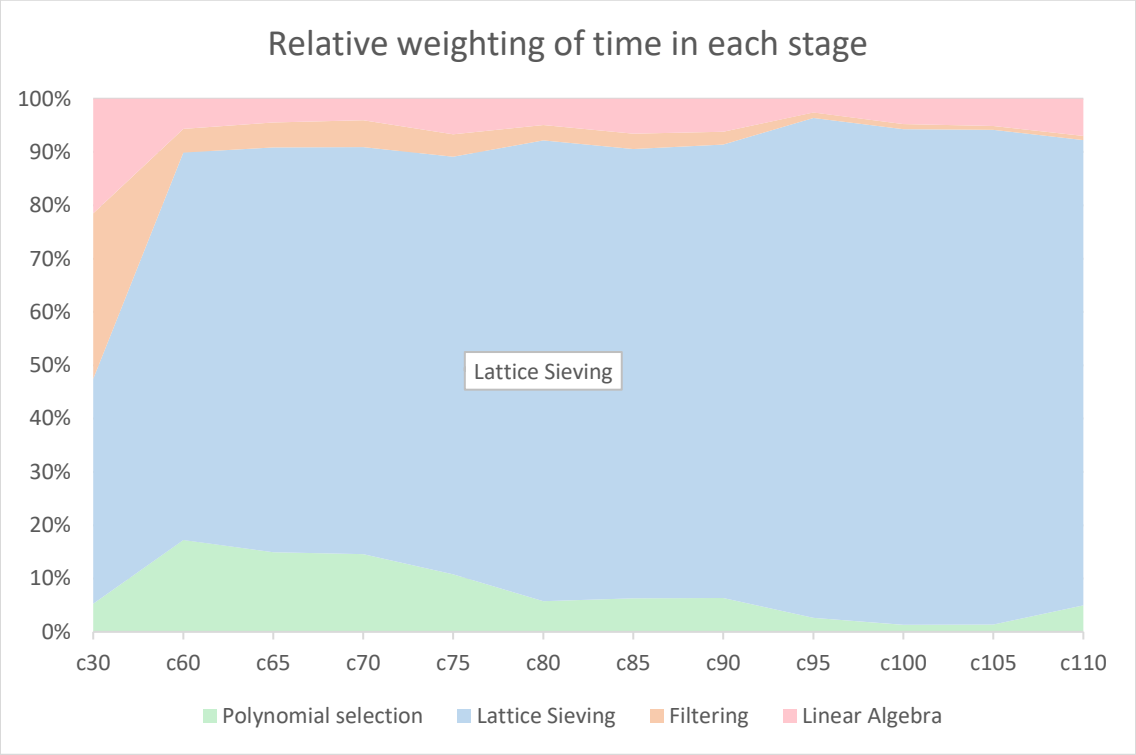


While the resulting MurphyE score of found polynomials decreases:



Results for each composite integer size are given on each Factor page and also summarized in an "Index of Results" in the Summary Results spreadsheet. The "Index of Results" tab also features graphs depicting execution time of a specific phase logarithmically in bar graphs as a convenience.

The relative weighting of effort across the different high-level stages (where complete factorization was performed) quickly shows the absolute dominance of Lattice Sieving:



Hopefully this data set and associated materials will meaningfully benefit future research in this area.

7 SUMMARY OF SYSTEMS USED

On-demand AWS Linux instances were used to perform these computations. Below is a summary of which systems were used for which job, their calculated CPU time, wall clock time, and which job they did (full factorization or polynomial selection only):

AWS instance type:	Quantity	Jobs	CPU time: Poly[1] [total] (seconds)	Wall clock (seconds)
t2.medium	1	c30 (full)	0.09 [3.51]	
		c60 (full)	7.19 [44.56]	
		c65 (full)	11.31 [81.90]	
		c70 (full)	17.17 [130.51]	
		c75 (full)	27.67 [273.45]	

		c80 (full)	28.46 [530.5]	
		c85 (full)	56.01 [948.69]	
		c90 (full)	112.92 [1880.20]	
t2.micro	25	c95 (full)	114.99 [4740.62]	461.40
		c100 (full)	135.85 [34457.86]	2306.82
		c105 (full)	369.99 [71901.06]	4445.65
c5.large	18	c110 (full)	1463.86 [30404.47]	3174.77
		c115 (poly)	1325.88	Not recorded
		c120 (poly)	1860.94	
		c125 (poly)	3403.97	
		c130 (poly)	9292.92	
		c135 (poly)	13510.46	
		c140 (poly)	35356.14	
		c145 (poly)	57793.93	
		c150 (poly)	76037.61	2417.83
		c155 (poly)	99722.24	2970.32
		c160 (poly)	228,181.4	6611.77
		c165 (poly)	712,519	20,010.91
		c170 (poly)	1,570,250	43,877.4
		c175 (poly)	4,063,750	118,806.6
		c180 (poly)	2,399,020	66,586.6
		c185 (poly)	1,099,447	31,718
		c190 (poly)	6,679,416	196,908.2

1 – Includes only Size optimization and Root optimization substeps for consistency.

8 FURTHER WORK

Many questions remain unanswered.

- What are the values for composites from 195 – 320 characters?
- What is the most efficient method to find good polynomials?
- Is there correlation between polynomials with a certain MurphyE score and subsequent phases? If so, how strong is that correlation?
- What are the optimal parameters for CADO-NFS?
- What is the density of polynomials of a given score as "incr" and "P" parameters vary?
- Why does c30 perform so well?
- Are there more efficient values of "incr"?
- Should "incr" be constant?
- How much variation is there between trials of any given substage?
- Does this change as the composite integer increases in size? If so, how?

ACKNOWLEDGEMENTS

Special thanks to Dr. Saurabh Shintre for reviews of early versions of this paper and extremely helpful feedback.

REFERENCES

Alpern, Dario
Integer factorization calculator
https://www.alpertron.com.ar/ECM.HTM

The CADO-NFS Development Team.
CADO-NFS, An Implementation of the Number Field Sieve Algorithm, Release 2.3.0, 2017
http://cado-nfs.gforge.inria.fr/

Kleinjung, Thorsten, Aoki, Kazumaro, Franke, Jens, Lenstra, Arjen K., Thome, Emmanuel, Bos, Joppe W., Gaudry, Pierrick, Et al.
Factorization of a 768-bit RSA modulus
https://eprint.iacr.org/2010/006.pdf

Murphy, Brian Antony
Polynomial Selection for the Number Field Sieve Integer Factorisation Algorithm
https://maths-people.anu.edu.au/~brent/pd/Murphy-thesis.pdf

Pomerance, Carl
A Tale of Two Sieves
http://www.ams.org/notices/199612/pomerance.pdf